

Problem Set 8: Packaging a Module and Automated Style Linting

Navneet Das

16-09-2026

Header

Course: Coding for AI

Assignment: Problem Set 8

Author: Navneet Das

Source Code

1. setup.py

```
1 from setuptools import setup
2
3 setup(
4     name="vsearch",
5     version="1.0",
6     description="Text search tools for AI preprocessing",
7     author="Navneet Das",
8     author_email="A26MTAI0017@bennett.edu.in",
9     url="localhost",
10    py_modules=["vsearch"]
11 )
```

2. vsearch.py (Refactored and PEP 8 Compliant)

```
1 def search4letters(phrase: str, letters="aeiou") -> set:
2     """Return the set of the letters found in the phrase."""
3     return set(letters).intersection(set(phrase))
```

Package Build and Installation (Screenshots)

```
D:\M.tech\coding for ai\Week-08\text_utility_pkg>py -3.13 -m pip install --upgrade build
Collecting build
  Using cached build-1.6.1-py3-none-any.whl.metadata (5.6 kB)
Collecting packaging>=24.0 (from build)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pyproject_hooks (from build)
  Using cached pyproject_hooks-1.3.0-py3-none-any.whl.metadata (1.3 kB)
Collecting colorama (from build)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Using cached build-1.6.1-py3-none-any.whl (31 kB)
Downloading packaging-26.3-py3-none-any.whl (129 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached pyproject_hooks-1.3.0-py3-none-any.whl (10 kB)
Installing collected packages: pyproject_hooks, packaging, colorama, build
Successfully installed build-1.6.1 colorama-0.4.6 packaging-26.3 pyproject_hooks-1.3.0
```

Figure 1: Upgrading the build module

```
D:\M.tech\coding for ai\Week-08\text_utility_pkg>py -3.13 -m build --sdist
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools >= 40.8.0
* Getting build dependencies for sdist...
running egg_info
creating vsearch.egg-info
writing vsearch.egg-info\PKG-INFO
writing dependency_links to vsearch.egg-info\dependency_links.txt
writing top-level names to vsearch.egg-info\top_level.txt
writing manifest file 'vsearch.egg-info\SOURCES.txt'
reading manifest file 'vsearch.egg-info\SOURCES.txt'
writing manifest file 'vsearch.egg-info\SOURCES.txt'
* Installed build dependency versions:
  - setuptools==84.0.0
* Building sdist...
running sdist
running egg_info
writing vsearch.egg-info\PKG-INFO
writing dependency_links to vsearch.egg-info\dependency_links.txt
writing top-level names to vsearch.egg-info\top_level.txt
reading manifest file 'vsearch.egg-info\SOURCES.txt'
writing manifest file 'vsearch.egg-info\SOURCES.txt'
running check
creating vsearch-1.0
creating vsearch-1.0\vsearch.egg-info
copying files to vsearch-1.0...
copying README.txt -> vsearch-1.0
copying setup.py -> vsearch-1.0
copying vsearch.py -> vsearch-1.0
copying vsearch.egg-info\PKG-INFO -> vsearch-1.0\vsearch.egg-info
copying vsearch.egg-info\SOURCES.txt -> vsearch-1.0\vsearch.egg-info
copying vsearch.egg-info\dependency_links.txt -> vsearch-1.0\vsearch.egg-info
copying vsearch.egg-info\top_level.txt -> vsearch-1.0\vsearch.egg-info
Writing vsearch-1.0\setup.cfg
Creating tar archive
removing 'vsearch-1.0' (and everything under it)
Successfully built vsearch-1.0.tar.gz
```

Figure 2: Building the distribution package (sdist)

```
D:\M.tech\coding for ai\Week-08\text_utility_pkg>py -3.13 -m pip install dist/vsearch-1.0.tar.gz
Processing .\dist\vsearch-1.0.tar.gz
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: vsearch
  Building wheel for vsearch (pyproject.toml) ... done
  Created wheel for vsearch: filename=vsearch-1.0-py3-none-any.whl size=1297 sha256=e220813919a40f9668407bb2f2ec1cf3c2cdca09a80bc5404792f15072cd03dc
  Stored in directory: c:\users\updes\appdata\local\pip\cache\wheels\ba\7d\e0\225a10b76c68614f3e1e44da5b6572b4d981b45db2f6dee5f1
Successfully built vsearch
Installing collected packages: vsearch
Successfully installed vsearch-1.0
```

Figure 3: Installing the package using pip

```
D:\M.tech\coding for ai\Week-08\text_utility_pkg>py -3.13
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import vsearch
>>> vsearch.search4letters("TGGACC","GC")
{'G', 'C'}
```

Figure 4: Verifying successful import in the Python interactive shell

Diagnostic Linting Output (Screenshots)

```
D:\M.tech\coding for ai\Week-08>py -3.13 -m pip install --upgrade pycodestyle
Collecting pycodestyle
  Downloading pycodestyle-2.14.0-py2.py3-none-any.whl.metadata (4.5 kB)
Downloading pycodestyle-2.14.0-py2.py3-none-any.whl (31 kB)
Installing collected packages: pycodestyle
Successfully installed pycodestyle-2.14.0
```

Figure 5: Upgrading the pycodestyle linter

```
D:\M.tech\coding for ai\Week-08\text_utility_pkg - v2>py -3.13 -m pycodestyle vsearch.py
vsearch.py:1:26: E231 missing whitespace after ':'
vsearch.py:1:39: E251 unexpected spaces around keyword / parameter equals
vsearch.py:1:41: E251 unexpected spaces around keyword / parameter equals
vsearch.py:3:1: W293 blank line contains whitespace
```

Figure 6: Before Fixes: Intentional PEP 8 flaws injected and detected

```
D:\M.tech\coding for ai\Week-08\text_utility_pkg - v2>py -3.13 -m pycodestyle vsearch.py
D:\M.tech\coding for ai\Week-08\text_utility_pkg - v2>_
```

Figure 7: After Fixes: Clean verification with exit code 0

Submission Note

Testing Package Installation & Import Scope:

I successfully navigated to a different directory (outside of `text_utility_pkg`) and ran the Python interactive shell. Importing `vsearch` and calling `vsearch.search4letters("TGGACC",`

"GC") returned {'C', 'G'}, confirming the package is globally accessible in my environment.

If I had run the interactive shell from inside the `text_utility_pkg` directory, Python would have simply found the local `vsearch.py` file immediately in the current working directory. The Python interpreter always checks the current directory first before searching the installed `site-packages`. Therefore, testing the import from within the source folder would mask any installation failures; the import would succeed even if the `pip install` step had completely failed, providing a false sense of success. Testing from an external directory guarantees that Python is genuinely loading the installed module from the system's package library.